

TEORÍA DE

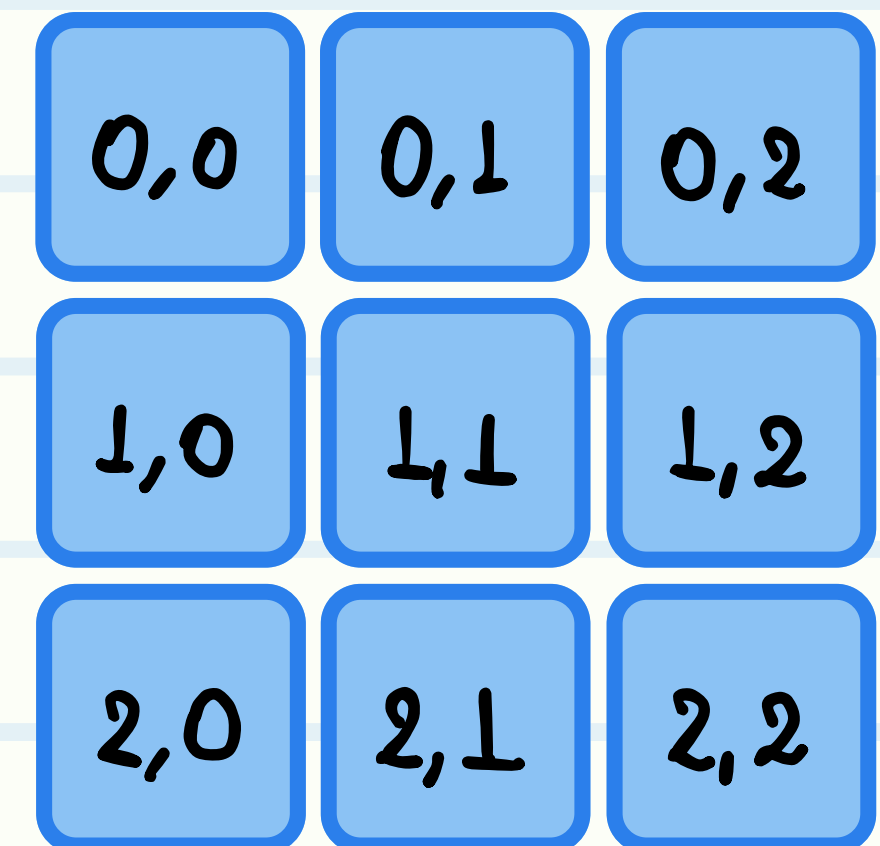
ESTRUCTURAS DE DATOS DEL STL

Presentado por CPC UNJFSC

ÍNDICE



- Arreglos estáticos (Array)
- Concepto de STL
- Arreglos dinámicos (Vector)
- Pilas (Stack)
- Colas (Queue)
- Colas de doble extremo (Deque)



ARREGLOS ESTÁTICOS (ARRAY)

Un array es una lista de elementos de un mismo tipo de dato.

Edades = 18 21 17 20 18

Nombres = “Jhon” “Zahir” “Miguel”

Su tamaño es estático, lo que quiere decir que una vez definida la cantidad de elementos del array, no puedes cambiarla. Ni añadir, ni quitar elementos.

Definimos nuestro arreglo “**edades**” con 5 elementos del tipo de dato “**int**”, es decir, solo almacenará 5 números enteros.

Edades = 18 21 17 20 18

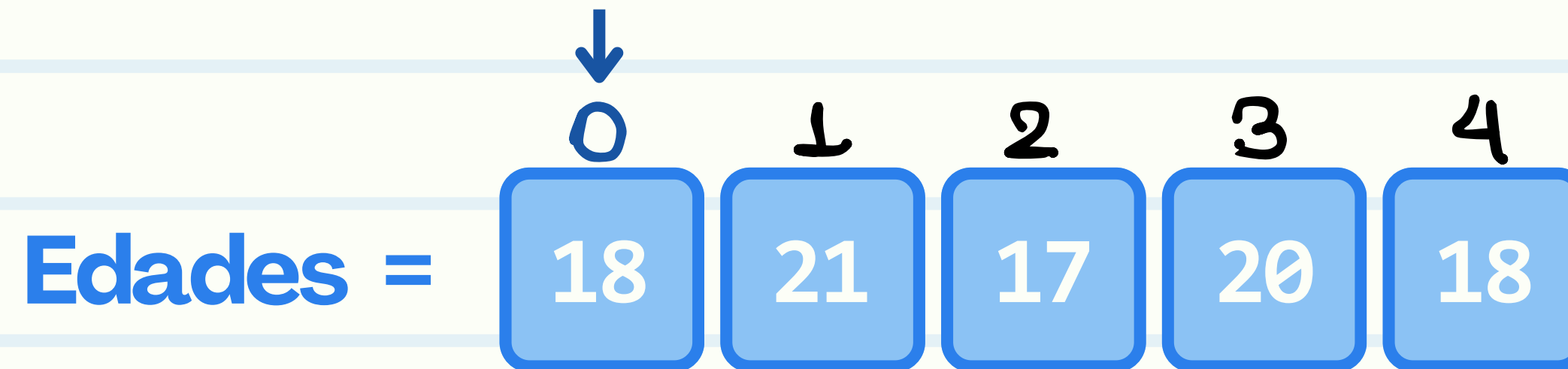
```
int edades[5] = {18, 21, 17, 20, 18};
```

Se debe respetar el tipo de dato y cantidad de elementos definidos, de lo contrario ocurrirá un **error de sintaxis**.

Nombres = “Jhon” “Zahir” “Miguel”

```
string nombres[3] = {"Jhon", "Zahir", "Miguel"};
```


Los índices son aquellos números que indican la posición de un elemento en un arreglo. El primer elemento de un array tendrá como índice 0.



Para acceder a un elemento en determinada posición debemos utilizar el **operador []** de acceso, especificando la posición deseada.

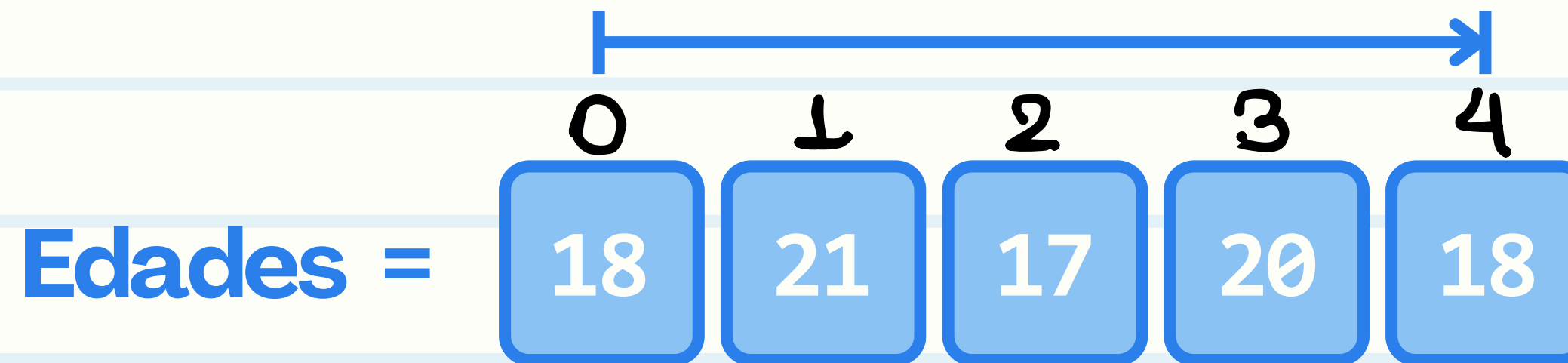
```
int edades[5] = {18, 21, 17, 20, 18};

int primera = edades[0];
int segunda = edades[1];
```

índice



Para recorrer cada elemento en un array de “n” elementos, debemos utilizar un bucle que empiece en el índice “0” (primer elemento) y vaya incrementando hasta el índice “n - 1” (último elemento).



```
int n = 5;
int edades[n] = {18, 21, 17, 20, 18};

for (int indice = 0; indice < n; indice++) {
    cout << edades[indice] << ' ';
}
```

ENCUENTRA EL ELEMENTO MÁS GRANDE EN UN ARREGLO

Dado un arreglo “A”, tenemos que encontrar el elemento más grande del arreglo.

ENTRADA

La primera línea de entrada indica un número entero “n” ($1 \leq n \leq 50$), la cantidad de elementos del arreglo.

La segunda línea contiene “n” números enteros “ A_i ” ($-10^4 \leq A_i \leq 10^4$), los elementos del arreglo.

SALIDA

Un único entero, el más grande del arreglo.

[Ir al problema](#)



Dos formas de solucionar este problema sin ordenar sus elementos.

```
5  int main() {
6      int casos; cin >> casos;
7      while (casos--) {
8          int n; cin >> n;
9          int numeros[n];
10
11         for (int i = 0; i < n; i++) {
12             cin >> numeros[i];
13         }
14
15         int mayor = INT_MIN;
16         for (int i = 0; i < n; i++) {
17             if (numeros[i] > mayor) {
18                 mayor = numeros[i];
19             }
20         }
21
22         cout << mayor << endl;
23     }
24
25     return 0;
26 }
```

```
5  int main() {
6      int casos; cin >> casos;
7      while (casos--) {
8          int n; cin >> n;
9          int numeros[n];
10
11         for (int i = 0; i < n; i++) {
12             cin >> numeros[i];
13         }
14
15         int mayor = INT_MIN;
16         for (int i = 0; i < n; i++) {
17             mayor = max(mayor, numeros[i]);
18         }
19
20         cout << mayor << endl;
21     }
22
23     return 0;
24 }
```

ENCUENTRA EL SEGUNDO ELEMENTO MÁS PEQUEÑO Y SEGUNDO MÁS GRANDE

Dado un arreglo “A” de “n” enteros diferentes, tenemos que encontrar el segundo elemento más pequeño y el segundo elemento más grande.

ENTRADA

La primera línea de entrada indica un número entero “n” ($4 \leq n \leq 50$), la cantidad de elementos del arreglo.

La segunda línea contiene “n” números enteros diferentes “ A_i ” ($-10^4 \leq A_i \leq 10^4$), los elementos del arreglo.

SALIDA

Imprime el segundo más pequeño seguido del segundo más grande.

[Ir al problema](#)




```

4
5 int main() {
6     int casos; cin >> casos;
7     while (casos--) {
8         int n; cin >> n;
9         int arr[n];
10        for (int i = 0; i < n; i++) {
11            cin >> arr[i];
12        }
13
14        int mayor = INT_MIN, menor = INT_MAX;
15        for (int i = 0; i < n; i++) {
16            if (arr[i] > mayor) {
17                mayor = arr[i];
18            }
19
20            if (arr[i] < menor) {
21                menor = arr[i];
22            }
23        }
24
25        int segMayor = INT_MIN, segMenor = INT_MAX;
26        for (int i = 0; i < n; i++) {
27            if (arr[i] > segMayor && arr[i] != mayor) {
28                segMayor = arr[i];
29            }
30
31            if (arr[i] < segMenor && arr[i] != menor) {
32                segMenor = arr[i];
33            }
34        }
35
36        cout << segMenor << ' ' << segMayor << '\n';
37    }
38

```

```

4
5 int main() {
6     int casos; cin >> casos;
7     while (casos--) {
8         int n; cin >> n;
9         int arr[n];
10        for (int i = 0; i < n; i++) {
11            cin >> arr[i];
12        }
13
14        int mayor = INT_MIN, segMayor = INT_MIN;
15        for (int i = 0; i < n; i++) {
16            if (arr[i] > mayor) {
17                segMayor = mayor;
18                mayor = arr[i];
19            } else if (arr[i] > segMayor && arr[i] != mayor) {
20                segMayor = arr[i];
21            }
22        }
23
24        int menor = INT_MAX, segMenor = INT_MAX;
25        for (int i = 0; i < n; i++) {
26            if (arr[i] < menor) {
27                segMenor = menor;
28                menor = arr[i];
29            } else if (arr[i] < segMenor && arr[i] != menor) {
30                segMenor = arr[i];
31            }
32        }
33
34        cout << segMenor << ' ' << segMayor << '\n';
35    }
36

```

Dos formas de solucionar este problema sin ordenar sus elementos.

STANDARD TEMPLATE LIBRARY (STL)

El STL (Biblioteca de Plantillas estándar), es un conjunto de plantillas de contenedores, clases y métodos.

Su objetivo principal es facilitar la implementación de diferentes estructuras de datos sin tener que crearlas desde cero.

CONTENEDORES:

- Vector
- Queue
- Stack
- Deque
- Priority Queue
- List
- Set
- Multiset
- Unordered Set
- Map
- Multimap
- Unordered Map

VECTORES

Un vector es una lista no ordenada de elementos de un mismo tipo de dato. Con la particularidad de que su tamaño es dinámico.

Podemos añadir elementos al final del vector.

Edades =



`.push_back(15)`



Podemos quitar el elemento del final del vector.

Edades =



`.pop_back()`

- Crear un vector

```
1 vector<int> v1; // Vector sin elementos (vacío)
2 vector<int> v2 = {1, 2, 3, 1}; // Con elementos definidos
3 vector<int> v3(10); // Con 10 elementos (todos son 0 por defecto)
4 vector<int> v4(8, 3); // Con 8 elementos (todos serán 3)
```

- Añadir/Quitar elementos al final del vector

```
1 vector<int> edades = {18, 21, 17, 20, 18};
2 edades.push_back(15); // Añadir elemento
3 edades.pop_back(); // Quitar elemento
```

- Propiedades del vector

```
1 cout << edades.size(); // Tamaño del vector
2 cout << edades.empty(); // "true" si está vacío
```

- Acceso

```
1  cout << edades[2]; // Imprime el tercer elemento
2  cout << edades.front(); // Primer elemento
3  cout << edades.back(); // Último elemento
```

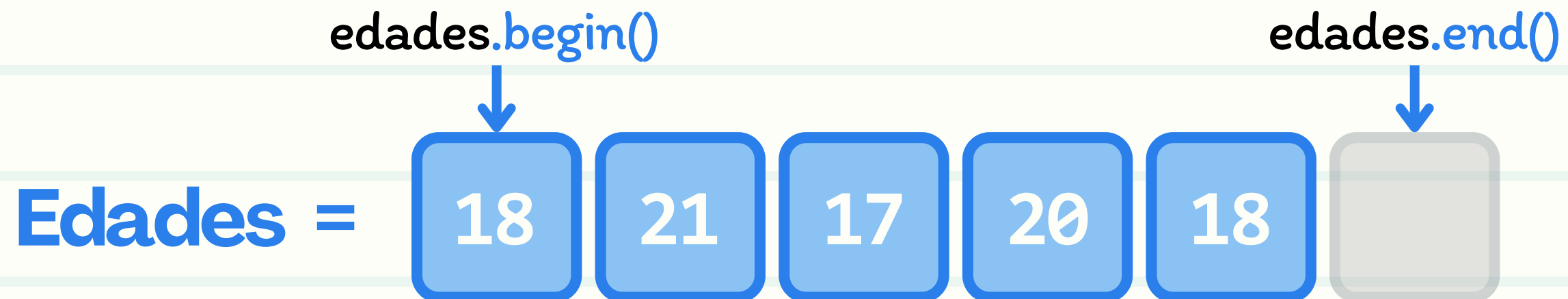
- Modificadores

```
1  edades.resize(10); // Cambia el tamaño a 10
2  edades.clear(); // Vacía el vector
```

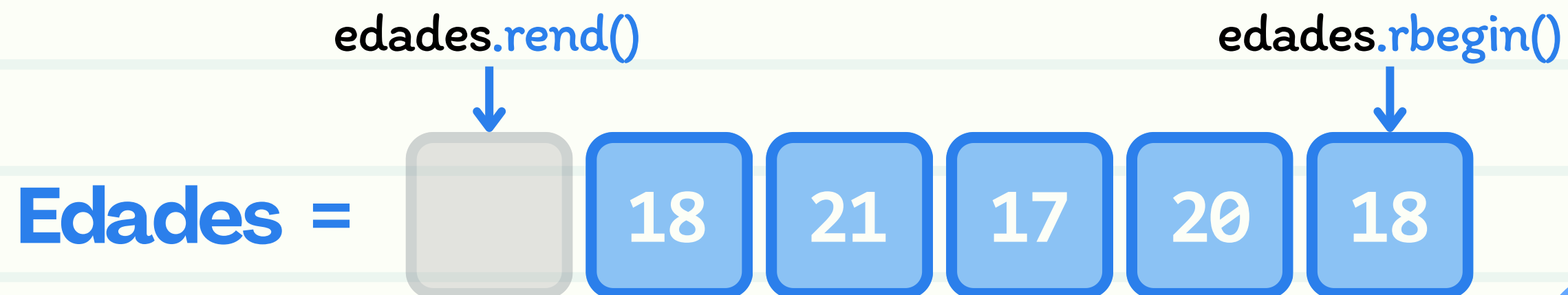
- Iteradores

```
1  vector<int>::iterator it1 = edades.begin();
2  vector<int>::iterator it2 = edades.end();
3  vector<int>::reverse_iterator it3 = edades.rbegin();
4  vector<int>::reverse_iterator it4 = edades.rend();
```

ITERADORES



ITERADORES INVERSOS



[Ver fuentes](#)



COMPROBAR SI LOS ELEMENTOS ESTÁN ORDENADOS

Dada una matriz tamaño n cuyos elementos son números enteros, escriba un programa para verificar si la matriz dada está ordenada (ascendente o descendente).

ENTRADA

La primera línea de entrada indica un número entero " n " ($1 \leq n \leq 50$), la cantidad de elementos del arreglo.

La segunda línea contiene " n " números enteros " A_i " ($-10^4 \leq A_i \leq 10^4$), los elementos del arreglo.

SALIDA

Si la matriz está ordenada, devuelve "YES", de lo contrario, devuelve "NO".

[Ir al problema](#)



```

5  int main() {
6      int casos; cin >> casos;
7      while (casos--) {
8          int n; cin >> n;
9          vector<int> v(n);
10         for (int i = 0; i < n; i++) {
11             cin >> v[i];
12         }
13
14         bool ascendente = true;
15         for (int i = 0; i < n-1; i++) {
16             if (v[i] > v[i+1]) {
17                 ascendente = false;
18                 break;
19             }
20         }
21
22         bool descendente = true;
23         for (int i = 0; i < n-1; i++) {
24             if (v[i] < v[i+1]) {
25                 descendente = false;
26                 break;
27             }
28         }
29
30         if (ascendente || descendente) {
31             cout << "YES" << endl;
32         } else {
33             cout << "NO" << endl;
34         }
35     }
36
37     return 0;
38 }

```

```

5  int main() {
6      int casos; cin >> casos;
7      while (casos--) {
8          int n; cin >> n;
9          vector<int> v(n);
10         for (int i = 0; i < n; i++) {
11             cin >> v[i];
12         }
13
14         if (is_sorted(v.begin(), v.end()) ||
15             is_sorted(v.rbegin(), v.rend())) {
16             cout << "YES" << endl;
17         } else {
18             cout << "NO" << endl;
19         }
20     }
21
22     return 0;
23 }

```



```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main() {
6      int casos; cin >> casos;
7      while (casos--) {
8          int n; cin >> n;
9          int arr[n];
10         for (int i = 0; i < n; i++) {
11             cin >> arr[i];
12         }
13
14         if (is_sorted(arr, arr+n) ||
15             is_sorted(arr, arr+n, greater<int>())) {
16             cout << "YES" << endl;
17         } else {
18             cout << "NO" << endl;
19         }
20     }
21
22     return 0;
23 }

```

En este problema, no es obligatorio usar vectores, pero el concepto es más fácil de entender con ellos.

Los iteradores son un tipo de punteros, pero no todos los punteros son iteradores.

Un array es un puntero que señala a su primer elemento.

STACK (PILA)

- Conocida como «pila», sigue el principio de "último en entrar, primero en salir" (LIFO, Last-In First-Out).
- Como una pila de libros: pones y quitas siempre desde arriba.
- Entre sus usos más básicos se encuentran la notación posfija y verificar si una secuencia de brackets está balanceada.



- Métodos más importantes

```
1  stack<int> s; // Pila sin elementos
2
3  cout << s.size(); // Tamaño de la pila
4  cout << s.empty(); // "true" si está vacía
5
6  cout << s.top(); // Elemento en lo alto de la pila
7
8  s.push(10); // Añadir encima
9  s.pop();    // Eliminar el elemento de encima
```

- No es posible recorrer los elementos de una pila.
- No tiene iteradores ni índices.

BALANCEAR PARÉNTESIS

Dadas cadenas de paréntesis, determina si cada secuencia de paréntesis está equilibrada.

Por ejemplo, { **[** (**]**) } no está equilibrado porque el par de corchetes encierra un único paréntesis de apertura no coincidente: (, y el par de paréntesis encierra un único corchete de cierre no coincidente: **]**

ENTRADA

La primera línea contiene una sola cadena de texto representando la secuencia de paréntesis.

SALIDA

Imprime “YES” si la secuencia está equilibrada o “NO” si no lo está.

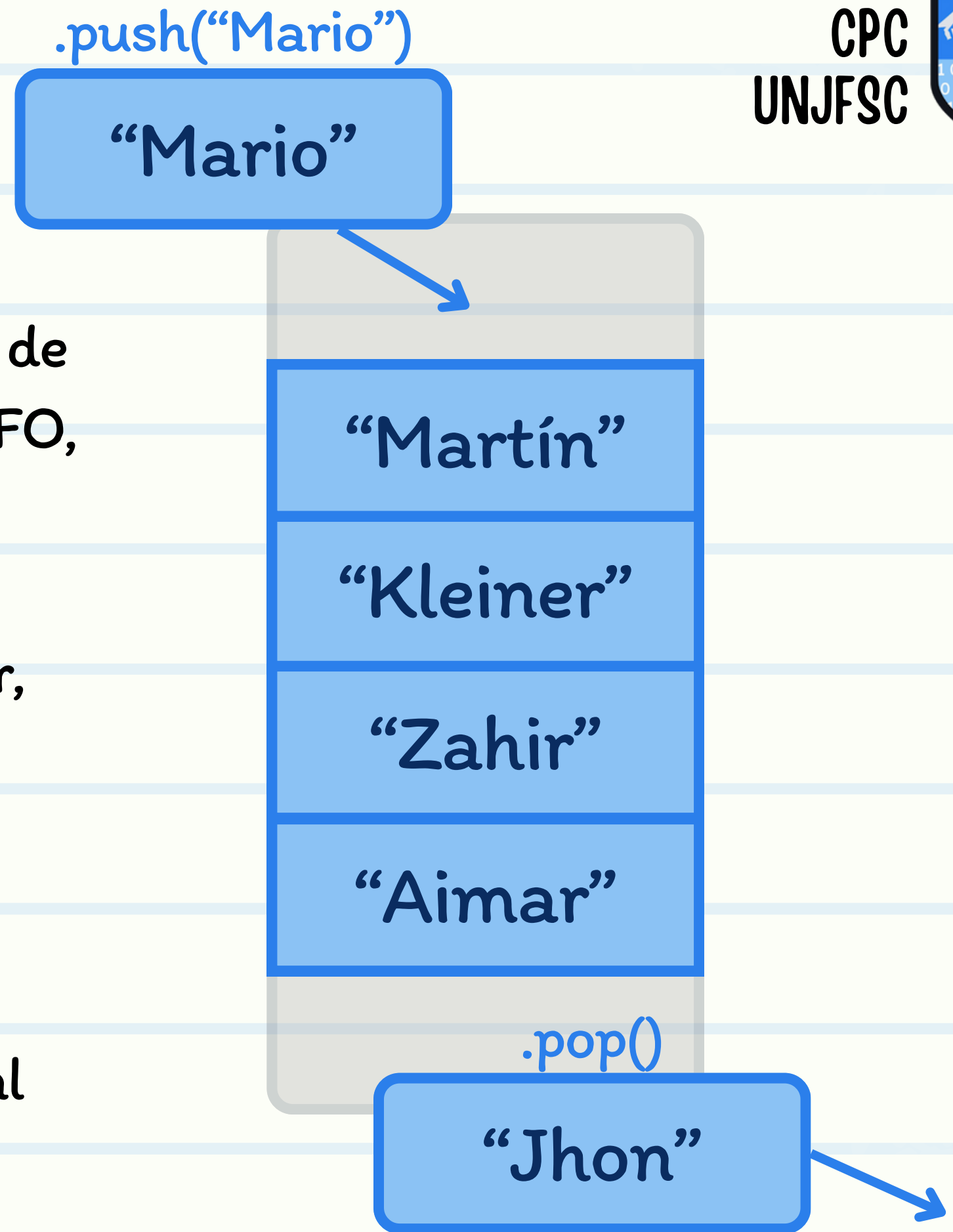
[Ir al problema](#)



```
1  int main() {
2      int casos; cin >> casos;
3      while (casos--) {
4          string s; cin >> s;
5          stack<char> q;
6          for (char curr : s) {
7              if (curr == '}' || curr == ']' || curr == ')') {
8                  if (q.empty() || (curr == '}' && q.top() != '{') ||
9                      (curr == ')') && q.top() != '(') ||
10                     (curr == ']' && q.top() != '[')) {
11                      cout << "NO" << endl;
12                      return;
13                  }
14                  q.pop();
15              } else {
16                  q.push(curr);
17              }
18          }
19
20          if (q.empty()) {
21              cout << "YES" << endl;
22          } else {
23              cout << "NO" << endl;
24          }
25      }
26
27      return 0;
28 }
```

QUEUE (COLA)

- Conocida como «cola», sigue el principio de “primero en entrar, primero en salir”(FIFO, First-In First-Out).
- Como en la cola para ingresar a un lugar, entras a la cola por el final y sales por delante.
- Uno de los usos más conocidos de esta estructura es en el algoritmo BFS, el cual se aplica a grafos.



- Métodos más importantes

```
1  queue<int> q; // Cola sin elementos
2
3  cout << q.size(); // Tamaño del vector
4  cout << q.empty(); // "true" si está vacío
5
6  cout << q.front(); // Primer elemento
7  cout << q.back(); // Último elemento
8
9  q.push(10); // Añadir al último
10 q.pop(); // Eliminar el primero
```

- No es posible recorrer los elementos de una cola.
- No tiene iteradores ni índices.

ROUND-ROBIN QUEUE

Hay "n" procesos en una cola. Cada proceso tiene un nombre y un tiempo de ejecución. El planificador round-robin atiende los procesos en el orden de la cola, asignando a cada uno un quantum (intervalo de tiempo). Si un proceso no termina dentro de su quantum, se interrumpe y se mueve al final de la cola para ser reanudado más tarde, y el planificador pasa al siguiente proceso.

ENTRADA

En la primera línea se dan dos enteros separados por un espacio: n - número de procesos y q: quantum en milisegundos.

En las siguientes "n" líneas se describen los procesos, uno por línea: nombre y tiempo

SALIDA

Para cada proceso, en el orden en que finaliza, imprime su nombre y el instante de tiempo (en ms) en que termina, separados por un espacio, uno por línea.

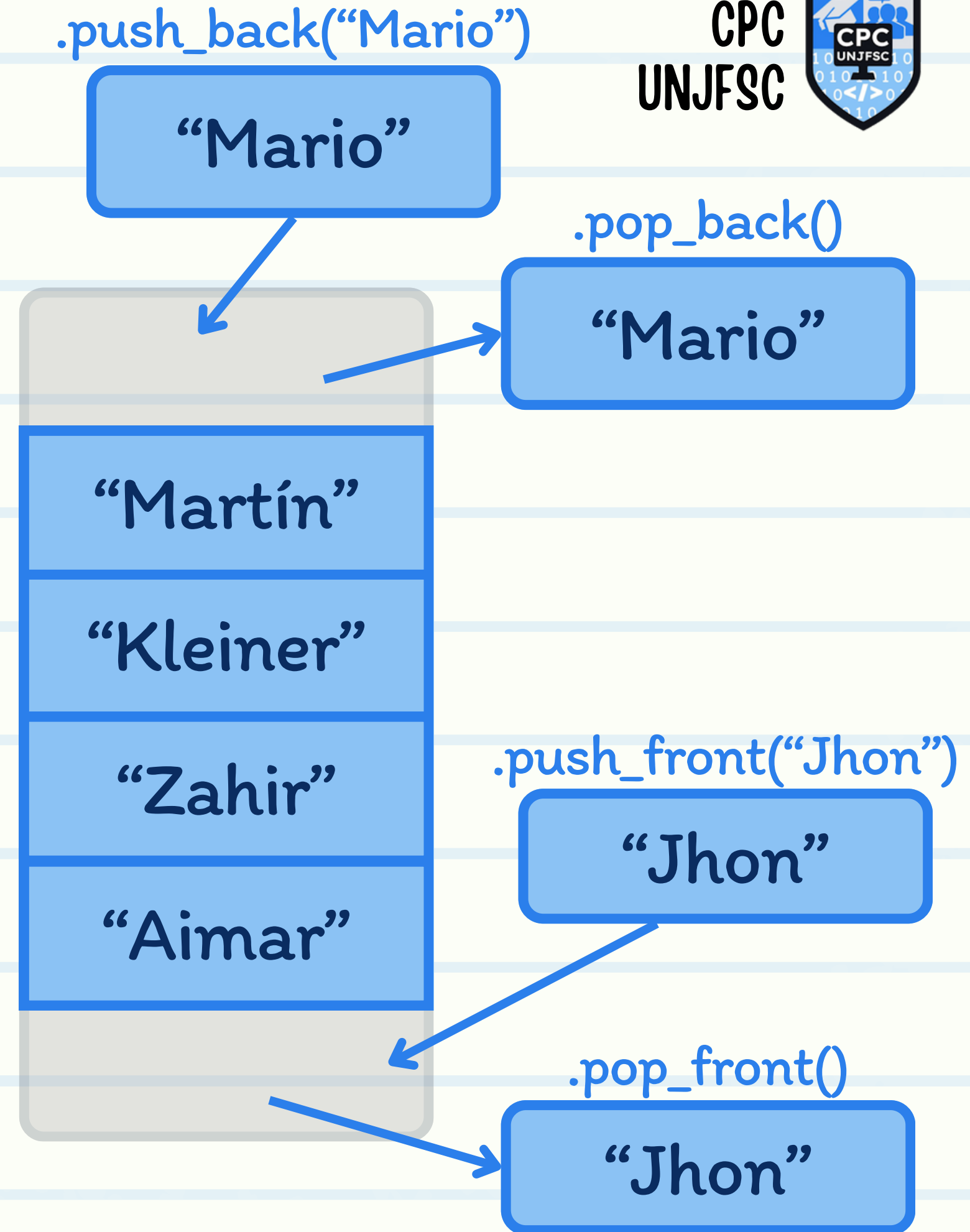
[Ir al problema](#)



```
1  int main() {
2      int casos; cin >> casos;
3      while (casos--) {
4          int n, ms; cin >> n >> ms;
5          queue<pair<string, int>> q;
6          while (n--) {
7              string name; cin >> name;
8              int score; cin >> score;
9              q.push({name, score});
10         }
11
12         int total = 0;
13         while (!q.empty()) {
14             auto [name, score] = q.front();
15             q.pop();
16
17             if (score <= ms) {
18                 total += score;
19                 cout << name << ' ' << total << '\n';
20             } else {
21                 q.push({name, score-ms});
22                 total += ms;
23             }
24         }
25     }
26
27     return 0;
28 }
```

DEQUE

- Es una estructura de datos que combina las características de las colas (queue) y pilas (stack).
- Permite la inserción y eliminación tanto al principio como al final de la secuencia, además también permite el acceso con el operador [] como un vector.
- Su nombre viene de “Double-Ended Queue” o Cola de doble final o doble extremo.



- Métodos más importantes

```
1  deque<int> d; // Deque sin elementos
2
3  cout << d.size();    // Tamaño del deque
4  cout << d.empty();   // "true" si está vacío
5
6  cout << d.front();   // Primer elemento
7  cout << d.back();    // Último elemento
8
9  d.push_back(10);     // Añadir al final
10 d.push_front(5);     // Añadir al inicio
11
12 d.pop_back();         // Eliminar el último
13 d.pop_front();        // Eliminar el primero
14
15 cout << d[0];        // Operador [ ]
16
17 deque<int>::iterator it = d.begin();
18 deque<int>::iterator it = d.end();
19 deque<int>::reverse_iterator it = d.rbegin();
20 deque<int>::reverse_iterator it = d.rend();
```

RED SOCIAL

Estás enviando mensajes en una de las redes sociales más populares desde tu smartphone. Tu smartphone puede mostrar como máximo "k" conversaciones más recientes con tus amigos. Inicialmente, la pantalla está vacía (es decir, el número de conversaciones mostradas es 0). Cada conversación es entre tú y alguno de tus amigos. Solo puede haber una conversación con cada amigo. Por lo tanto, cada conversación se define únicamente por el ID de tu amigo.

ENTRADA

La primera línea de la entrada contiene dos enteros n y k : el número de mensajes y el número de conversaciones que tu smartphone puede mostrar.

La segunda línea de la entrada contiene n enteros $id[1]$, $id[2]$, ..., $id[n]$, donde $id[i]$ es el ID del amigo que te envía el mensaje número i .

SALIDA

En la primera línea de la salida, imprime el número de conversaciones mostradas después de recibir todos los mensajes.

En la segunda línea, imprime m enteros, las ID de tus conversaciones.

[Ir al problema](#)


```
1  int main() {
2      int casos; cin >> casos;
3      while (casos--) {
4          int n, k; cin >> n >> k;
5          deque<int> show;
6          for (int i = 0; i < n; i++) {
7              int curr; cin >> curr;
8
9              auto existe = find(show.begin(), show.end(), curr);
10             if (existe != show.end()) continue;
11
12             if (show.size() ≥ k) show.pop_back();
13             show.push_front(curr);
14         }
15
16         cout << show.size() << '\n';
17         for (int& i : show) {
18             cout << i << ' ';
19         }
20         cout << '\n';
21     }
22
23     return 0;
24 }
```


Característica	Vector	Queue	Stack	Deque
¿Qué es?	Lista no ordenada	Cola (FIFO)	Pila (LIFO)	Cola de doble extremo
Constructor	<code>vector<T></code> <code>vector<T>(N)</code>	<code>queue<T></code>	<code>stack<T></code>	<code>deque<T></code> <code>deque<T>(N)</code>
Recorrido	Con índices e iteradores	×	×	Con índices e iteradores
Acceso rápido	Operador [] <code>.front()</code> <code>.back()</code>	<code>.front()</code> <code>.back()</code>	<code>.top()</code>	Operador [] <code>.front()</code> <code>.back()</code>
Añadir elementos	<code>.push_back()</code> <code>.pop_back()</code>	<code>.push()</code> ← final <code>.pop()</code> → inicio	<code>.push()</code> ← final <code>.pop()</code> → final	<code>.push_front()</code> <code>.push_back()</code> <code>.pop_front()</code> <code>.pop_back()</code>
Uso típico	Almacenar y recorrer varios datos	Procesar tareas en orden de llegada	Deshacer/rehacer acciones	Cuando necesitas tanto el funcionamiento de una cola como el de una pila

MUCHAS

GRACIAS

CPC UNJFSC
by zlarosav